

Reviewer Pack — Forman-Ricci Negative-Edge Count Lower-Bounds Tseitin DPLL S...

Entry a687b7685342 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 11:22:23 UTC

Forman-Ricci Negative-Edge Count Lower-Bounds Tseitin DPLL Size

Entry ID: a687b7685342 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 11:22:23 UTC

1. Conjecture

Field A (mathematical branch): Discrete Ricci curvature à la Forman (Forman 1998, ‘Bochner’s method for cell complexes’): the augmented combinatorial Ricci curvature $F(e)$ of an edge in a graph, computed in closed form from triangle and 4-cycle counts at e , distinct from Ollivier-Ricci (no transport LP) and essentially absent from proof complexity (≈ 0 papers, vs. Skenderi 2025 for Ollivier-Ricci on SAT) **Field B** (complexity object): DPLL / tree-resolution refutation size $T_DPLL(Tseitin(G, \sigma))$ of 3-regular Tseitin formulas with odd vertex charge $\sigma: V \rightarrow \mathbb{F}_2$ (the Urquhart / Ben-Sasson-Wigderson canonical sub-Frege gateway target)

Statement:

For a 3-regular graph $G=(V,E)$ and odd charge $\sigma: V \rightarrow \mathbb{F}_2$, define the augmented Forman-Ricci curvature on edge $e=(u,v)$ by $F(e) := 2 \cdot t(e) + 2 \cdot q(e) - (\deg(u) + \deg(v) - 2)$, where $t(e)$ is the number of triangles through e and $q(e)$ is the number of induced 4-cycles through e . Let $N_-(G) := \#\{e \in E : F(e) < 0\}$. We conjecture absolute constants $c \geq 0.4$ and $C \leq 5$ such that $\log_2 T_DPLL(Tseitin(G, \sigma)) \geq c \cdot N_-(G) - C$ for every odd-charged 3-regular Tseitin instance with $|V| \geq 12$, where T_DPLL counts recursive calls under unit propagation and a fixed lex variable order; one instance with $\log_2 T_DPLL < c \cdot N_-(G) - C$ falsifies it.

Rationale (proposer’s reasoning):

Negative Forman-Ricci on an edge encodes local ‘tree-like’ branching geometry — neither triangles nor short squares can short-circuit width-building inferences across e — so the count $N_-(G)$ is a poly-time, S_n -natural proxy for the ‘expander mass’ that powers Ben-Sasson-Wigderson width lower bounds, but read off from purely local edge statistics rather than λ_2 or Cheeger flow. Because Forman curvature decouples cleanly from spectral and metric-embedding invariants already attempted on Tseitin (Laplacian gap, Ollivier-Ricci, Ihara zeta, Leighton-Rao), an empirical correlation would expose a new local-to-global handle on the Urquhart target. The bridge is barrier-safe: the invariant is graph-only (no truth-table access), so no natural-proofs largeness, and it relativizes/algebrizes trivially.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 46969f981dbd2ab1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.85	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_prime(n):
        if n <= 1:
            return False
        for i in range(2, int(math.sqrt(n)) + 1):
            if n % i == 0:
                return False
        return True

    def generate_primes(n):
        primes = []
        num = 2
        while len(primes) < n:
            if is_prime(num):
                primes.append(num)
            num += 1
        return primes

    def generate_3_regular_graph(n):
```

```

graph = [[] for _ in range(n)]
degrees = [0] * n
added_edges = set()

for _ in range(3 * n // 2):
    while True:
        u, v = random.sample(range(n), 2)
        if (u, v) not in added_edges and (v, u) not in added_edges and degrees[u] < 3 and degrees[v] < 3:
            graph[u].append(v)
            graph[v].append(u)
            added_edges.add((u, v))
            degrees[u] += 1
            degrees[v] += 1
            break

    return graph

def generate_odd_charge(n):
    charge = [0] * n
    if sum(charge) % 2 == 0:
        charge[random.randint(0, n - 1)] ^= 1
    return charge

def bfs(graph, start):
    queue = [start]
    visited = set([start])
    while queue:
        node = queue.pop(0)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return len(visited) - 1

def count_triangles(graph, u):
    triangles = 0
    for v in graph[u]:
        for w in graph[v]:
            if w != u and w not in graph[u]:
                triangles += 1
    return triangles // 3

def count_4_cycles(graph, u):
    cycles = 0
    for v in graph[u]:
        for w in graph[v]:
            if w != u:
                for x in graph[w]:
                    if x != u and x not in graph[u] and x != v:
                        cycles += 1
    return cycles // 4

def forman_ricci_curvature(graph, charge, edge):
    u, v = edge
    t_e = count_triangles(graph, u) + count_triangles(graph, v)
    q_e = count_4_cycles(graph, u) + count_4_cycles(graph, v)

```

```

    return 2 * t_e + 2 * q_e - (len(graph[u]) + len(graph[v]) - 2)

def dpll(cnf):
    stack = []
    assignment = [None] * len(cnf)

    def propagate():
        while stack:
            literal, value = stack.pop()
            if assignment[literal] is None:
                assignment[literal] = value
            elif assignment[literal] != value:
                return False
        return True

    def unit_propagate():
        changed = True
        while changed:
            changed = False
            for clause in cnf:
                unsatisfied = [l for l, v in enumerate(clause) if assignment[l] is None and v
                                satisfied = any(assignment[l] == (v == -1) for l, v in enumerate(clause))
                if len(unsatisfied) == 1 and not satisfied:
                    literal, value = unsatisfied[0], 1
                    stack.append((literal, value))
                    assignment[literal] = value
                    changed = True

    def pure_literal_elimination():
        for literal in range(len(cnf)):
            positive_count = sum(1 for clause in cnf if literal in clause)
            negative_count = sum(1 for clause in cnf if -literal in clause)
            if positive_count == 0:
                assignment[literal] = True
            elif negative_count == 0:
                assignment[literal] = False

    def backtrack():
        while stack:
            literal, value = stack.pop()
            assignment[literal] = None
            for l, v in enumerate(cnf):
                if literal in v and assignment[l] is None:
                    stack.append((l, -v[l]))
                    break

    def solve():
        propagate()
        unit_propagate()
        pure_literal_elimination()
        if all(assignment[l] is not None for l in range(len(cnf))):
            return True
        literal = next(l for l in range(len(cnf)) if assignment[l] is None)
        stack.append((literal, 1))
        if solve():
            return True

```

```

        stack.pop()
        stack.append((literal, -1))
        if solve():
            return True
        backtrack()
        return False

    return solve()

def tseitin_encoding(graph, charge):
    n = len(graph)
    cnf = []

    for i in range(n):
        cnf.append([i * 2 + charge[i], -(i * 2 + 1 - charge[i])])

    for u in range(n):
        for v in graph[u]:
            if u < v:
                cnf.append([-u * 2 - charge[u], -v * 2 - charge[v], u * 2 + 1 - charge[u], v * 2 + 1 - charge[v]])

    return cnf

def log2_tseitin_size(cnf):
    size = sum(1 for clause in cnf if len(clause) > 0)
    return math.log2(size)

n_values = [12, 14, 16, 18, 20]
results = []

for n in n_values:
    graph = generate_3_regular_graph(n)
    charge = generate_odd_charge(n)
    cnf = tseitin_encoding(graph, charge)

    N_minus = sum(1 for u in range(n) for v in graph[u] if forman_ricci_curvature(graph, charge, u, v) > 0)
    T_DPLL = dpLL(cnf)
    log2_T_DPLL = log2_tseitin_size(cnf)

    results.append({
        "metric_name": "N_minus",
        "metric_value": N_minus,
        "instances_tested": 1,
        "conjecture_holds": log2_T_DPLL >= 0.4 * N_minus - 5,
        "counterexample": ""
    })

mean_N_minus = sum(result["metric_value"] for result in results) / len(results)
std_N_minus = math.sqrt(sum((result["metric_value"] - mean_N_minus) ** 2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

return {
    "mean_N_minus": mean_N_minus,
    "std_N_minus": std_N_minus,
    "support_fraction": support_fraction
}

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or generate_primes(30)

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"mean_N_minus\": {result['mean_N_minus']:.2f}, \"std_N_minus\": {result['std_N_minus']:.2f}}}")

    if result["support_fraction"] >= 0.8:
        print(f"RESULT: SUPPORTED mean={result['mean_N_minus']:.2f} std={result['std_N_minus']:.2f}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"N_minus < 0.4 * N_minus - 5\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results; pre-registered support condition cannot be verified | next: Increase timeout duration and re-run with stricter resource limits

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	108533
2	preregistration	ollama_remote	qwen3:8b	0	0	24233
3	novelty	ollama_remote	qwen3:8b	0	0	20771
4	novelty	ollama_remote	qwen3:8b	0	0	17162
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20787
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20642
7	judge	ollama_remote	qwen3:8b	0	0	14275

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 226401 ms total latency. Provider mix: {'claude_max': 1, 'ollama_remote': 6}

(full prompt+response transcripts available in research/audit/a687b7685342.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/a687b7685342.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/a687b7685342.tar.gz (if generated)